UNIVERSIDADE FUMEC

CURSO DE CIÊNCIA DA COMPUTAÇÃO

DESENVOLVIMENTO WEB FRONT-END

PROF. DÂNGELO SILVA MIRANDA

KAIQUE FERREIRA PEIXOTO

CARDVAULT JP

Trabalho 01 — Protótipo

Belo Horizonte

2026

1. NOME DA APLICAÇÃO

CardVault JP — plataforma web para gestão, catalogação, avaliação e negociação de cartas colecionáveis.

2. OBJETIVO

O objetivo da aplicação é oferecer a colecionadores e lojistas um ambiente único para catalogar e avaliar cartas colecionáveis de Pokémon TCG, com foco em coleções japonesas e itens de nicho. A plataforma permite consultar o acervo com filtros, visualizar detalhes de cada carta (variante, idioma, estado de conservação e histórico de preço), iniciar negociações e, para o administrador, manter o cadastro de cards e acompanhar indicadores por meio de um dashboard.

A aplicação será desenvolvida em Angular, consumindo uma API REST simulada com JSON-Server, com controle de acesso por Route Guards, carregamento sob demanda de módulos (Lazy Loading) e versionamento com GitFlow e Conventional Commits.

3. PRINCIPAIS FUNCIONALIDADES

Código	Requisito funcional	Descrição
RF01	Listagem de cards	Catálogo em grade com busca por nome e filtros por raridade, tipo e coleção.
RF02	Detalhes do card	Página com imagem ampliada, atributos, variante/edição, idioma, estado de conservação, histórico de preço e botão "Negociar".
RF03	Formulários com validação	Login, cadastro de usuário, cadastro/edição de cards e contato com validação de campos obrigatórios, formato de e-mail, tamanho de senha e valores numéricos.
RF04	Route Guards	Proteção das rotas administrativas (cadastro e dashboard) conforme o perfil autenticado (Cliente ou Administrador).
RF05	Lazy Loading	Módulos/rotas carregados sob demanda (ex.: área administrativa e dashboard), reduzindo o tamanho do carregamento inicial.
RF06	Dashboard	KPIs (total no acervo, vendas no mês, valor estimado) e gráficos de estoque por coleção, distribuição por raridade e vendas por mês.

RF07	API REST via JSON-Server	Operações de CRUD (GET, POST, PUT, DELETE) de cards, usuários e negociações sobre um arquivo `db.json`.
RF08	Git / GitFlow / Conventional Commits	Versionamento no GitHub com branches main, develop e feature/* e mensagens de commit padronizadas.

4. IDENTIDADE VISUAL

A identidade visual adota um tema escuro elegante, que valoriza as cores das cartas e remete a vitrines de itens de coleção.

Elemento	Valor	Uso
Fundo	#0F1220	Plano de fundo das páginas
Superfícies	#1A1F35	Navbar, painéis, cards e formulários
Destaque	#E3350D	Botões principais, links ativos, erros de validação
Dourado	#F2C14E	Raridades (Ultra Rara, Secreta) e valores monetários
Texto	#F5F5F7	Texto principal

Tipografia: Inter (Google Fonts), pesos 400 a 800. **Logo:** texto "CardVault JP" acompanhado de um ícone original em SVG de duas cartas sobrepostas, sem uso de marcas ou personagens oficiais; as cartas do protótipo são retângulos genéricos com gradiente e nomes fictícios. **Layout:** navbar fixa no topo (logo à esquerda; Catálogo, Dashboard, Sobre e Login à direita), conteúdo em grade, cantos arredondados (botões com 8 px) e resolução de referência desktop de 1440×900 px.

5. TELAS DO PROTÓTIPO

5.1 Login e Cadastro (login.html)

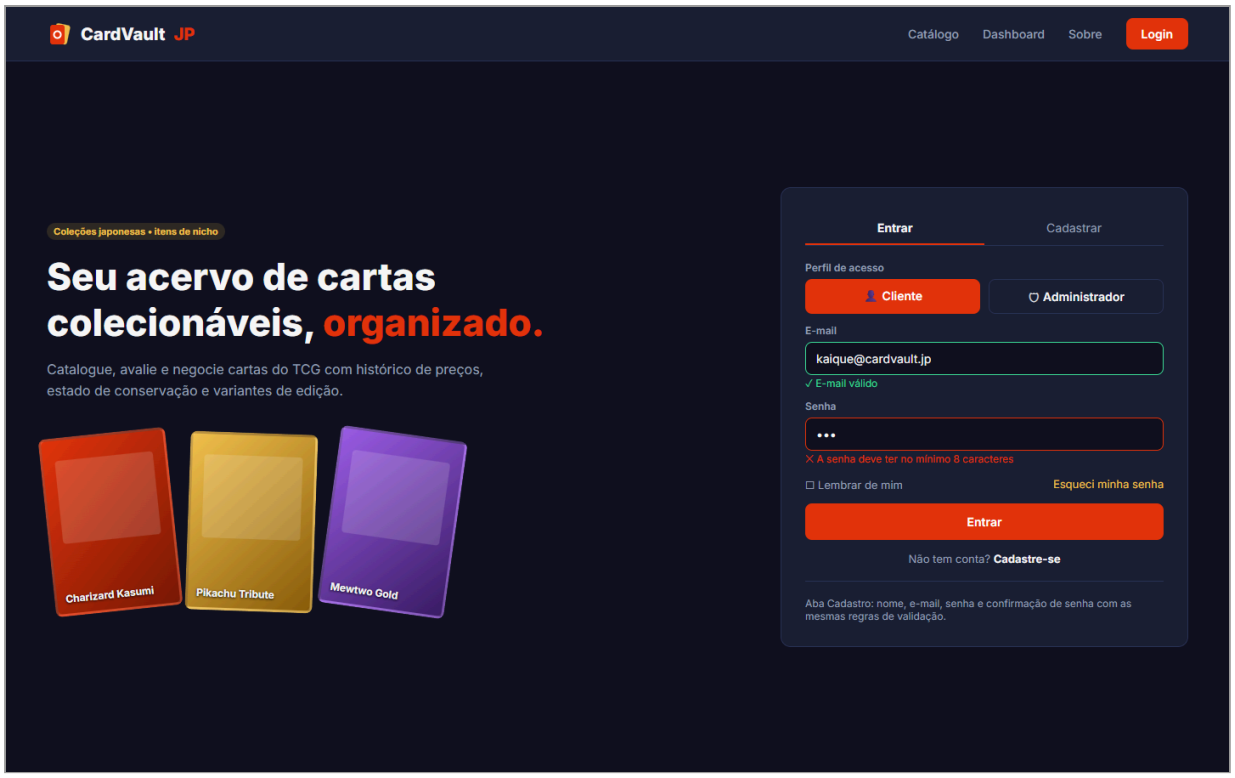


Figura 1 — Tela de Login e Cadastro

Tela de entrada com abas "Entrar" e "Cadastrar" e seletor de perfil (Cliente ou Administrador), que definirá as rotas liberadas pelos Route Guards. Apresenta validação visual: campo válido em verde e campo inválido em vermelho com mensagem explicativa (senha com menos de 8 caracteres).

5.2 Catálogo (catalogo.html)

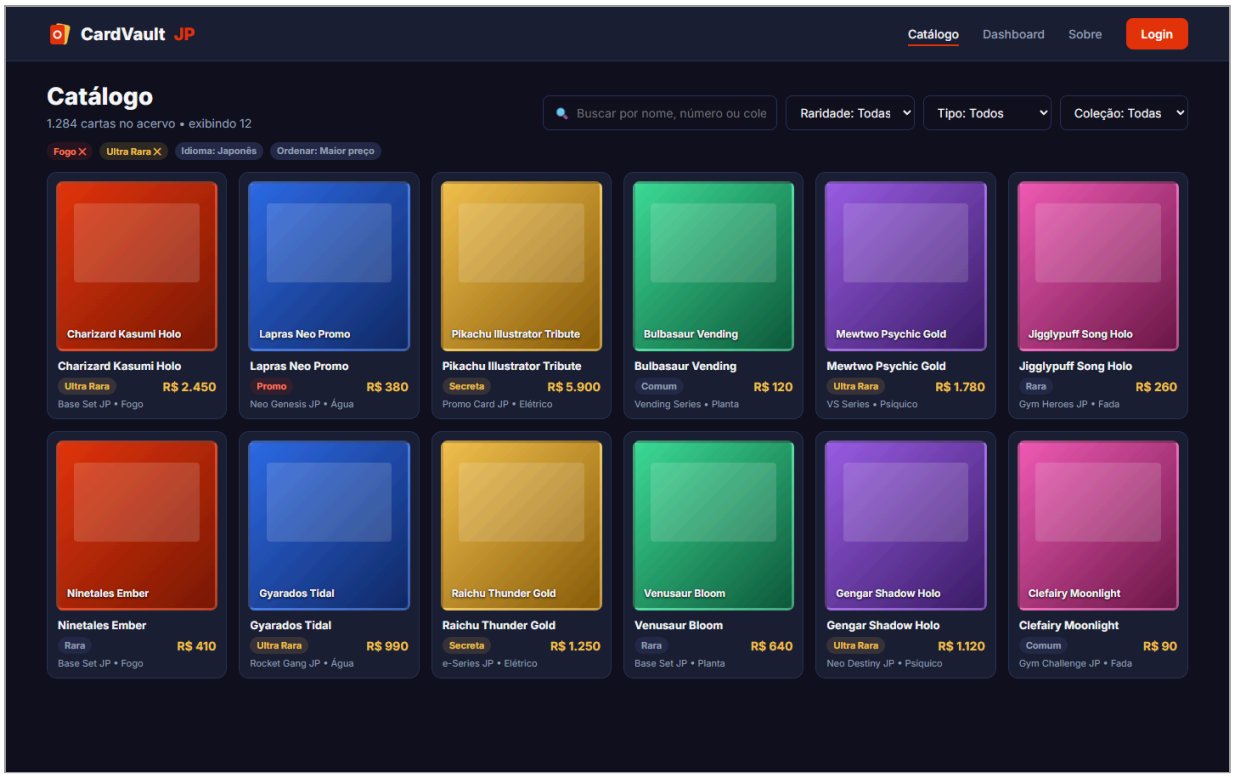


Figura 2 — Tela de Catálogo

Grade com 12 cards exibindo nome, raridade, preço, coleção e tipo. Possui campo de busca, filtros por raridade, tipo e coleção e indicação dos filtros ativos. Os dados virão do endpoint `GET /cards` do JSON-Server.

5.3 Detalhes do card (detalhe.html)

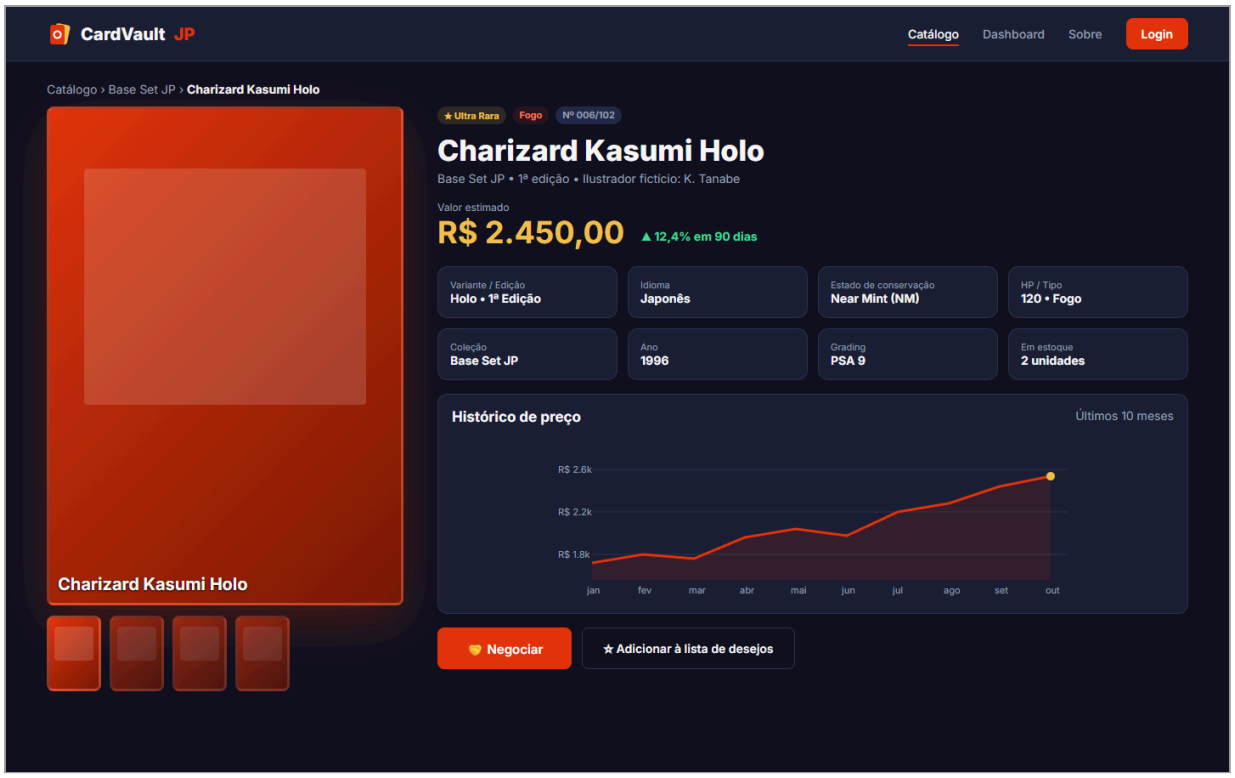


Figura 3 — Tela de Detalhes do Card

Exibe a imagem ampliada da carta e miniaturas, atributos (variante/edição, idioma, estado de conservação, HP/tipo, coleção, ano, grading e estoque), valor estimado com variação, gráfico SVG de histórico de preço e o botão "Negociar", que inicia uma proposta de negociação.

5.4 Cadastro administrativo (admin-cadastro.html)

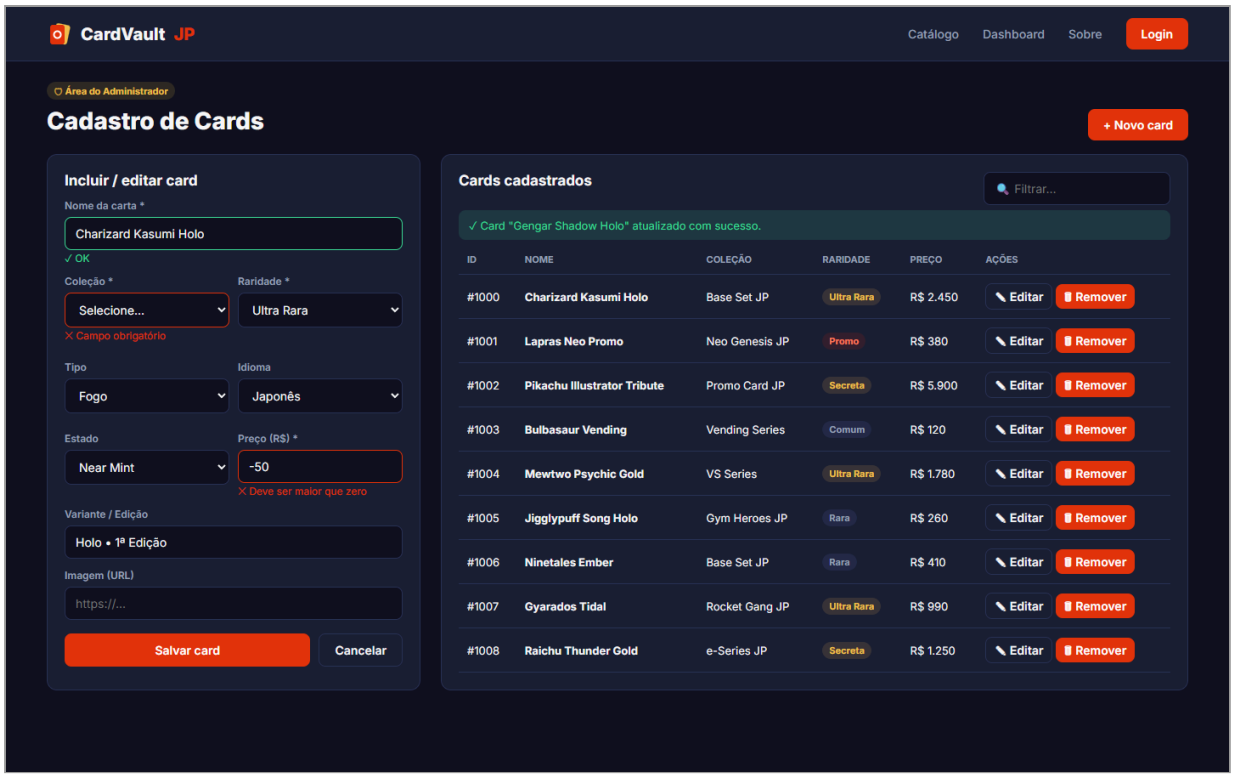


Figura 4 — Tela de Cadastro Administrativo

Área restrita ao Administrador. Contém formulário de inclusão/edição de card com mensagens de validação (campo obrigatório e preço maior que zero) e tabela de cards cadastrados com ações de editar e remover, além de mensagem de sucesso após a operação.

5.5 Dashboard (dashboard.html)



Figura 5 — Tela de Dashboard

Apresenta KPIs (total no acervo, vendas no mês, valor estimado do acervo e negociações abertas), gráfico de barras de estoque por coleção, gráfico de rosca da distribuição por raridade e gráfico de linha de vendas por mês, todos feitos em SVG/CSS, sem bibliotecas externas.

5.6 Sobre / Institucional (sobre.html)

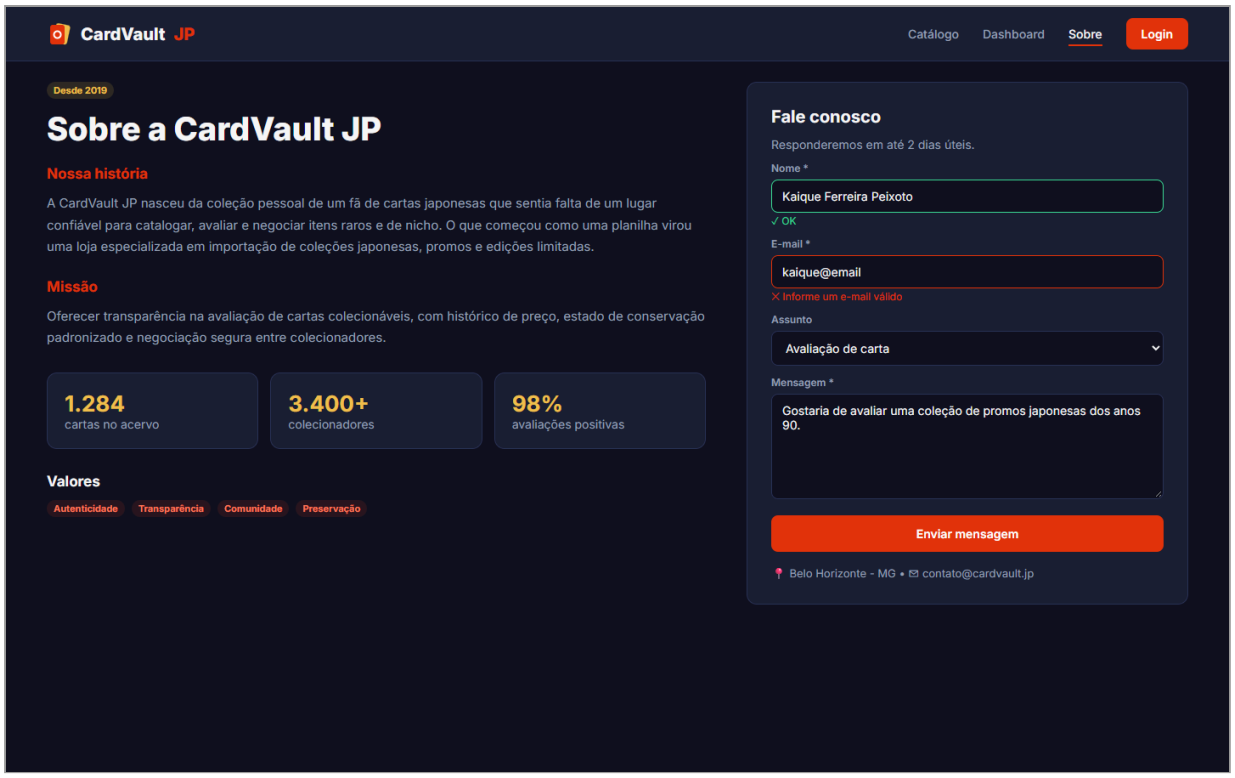


Figura 6 — Tela Sobre

Página institucional com a história da loja, missão, valores e números, além de formulário de contato com validação (nome, e-mail, assunto e mensagem).

6. INTEGRANTES

Kaique Ferreira Peixoto (trabalho individual).

7. REPOSITÓRIO E VERSIONAMENTO

Repositório: https://github.com/kaiquepeixoto/cardvault-jp

GitFlow: a branch `main` guarda apenas versões estáveis entregues; a `develop` integra o trabalho em andamento; cada funcionalidade é desenvolvida em uma branch `feature/*` criada a partir da develop (ex.: `feature/prototipo`) e integrada de volta com merge `--no-ff`, preservando o histórico da funcionalidade.

Conventional Commits: as mensagens seguem o formato `tipo(escopo): descrição`, em que o tipo indica a natureza da mudança (`feat` nova funcionalidade, `fix` correção, `docs` documentação, `style`, `refactor`, `chore` tarefas de manutenção). Commits usados neste trabalho:

- `feat(prototipo): adiciona tela de login e cadastro`
- `feat(prototipo): adiciona tela de catálogo`
- `feat(prototipo): adiciona tela de detalhes do card`

- `feat(prototipo): adiciona tela de cadastro administrativo`
- `feat(prototipo): adiciona tela de dashboard`
- `feat(prototipo): adiciona tela institucional`
- `docs(relatorio): adiciona relatório do trabalho 01`
- `chore(kanban): atualiza kanban do trabalho 01`

8. LINK DO PROTÓTIPO

Link Figma: _____

Arquivos HTML/CSS do protótipo: pasta `prototipo/` do repositório.